

Initial Setup of Jetson Orin Nano

<https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit#intro>

SSH - to use jetson's terminal on your laptop

```
sudo systemctl enable ssh
sudo systemctl start ssh
ip a or ip addr —to check ip address
ssh-keygen -R ip address — to remove prev key ( on laptop )
```

Then on your windows powershell do:

ssh desktop-name@ip-address, then enter the password of jetson
eg) ssh rabbit@172.13.20.04

Camera Setup (IMX 477 Arducam)

```
sudo apt update
sudo apt upgrade
sudo /opt/nvidia/jetson-io/jetson-io.py
```

This opens the **Jetson-IO Configuration Tool**

You'll see a menu like this:

```
Jetson-IO
├─ Configure Jetson 40pin Header
├─ Configure CSI Connector
└─ Save and reboot
```

Choose:

Configure CSI Connector

Then:

- Select **Camera**
- Select **IMX477 C** (or Arducam IMX477 if listed)
- Enable it

Finally:

Save and reboot

Reboot is mandatory

To open camera

```
gst-launch-1.0 nvarguscamerasrc ! nvvidconv ! autovideosink
gst-launch-1.0 -e \
```

To record a video (saved as dataset_raw_640.mp4)

```
nvarguscamerasrc sensor-id=0 ! \
'video/x-raw(memory:NVMM),width=1280,height=720,format=NV12,framerate=30/1' ! \
nvvidconv left=280 right=1000 top=0 bottom=720 ! \
'video/x-raw(memory:NVMM),width=640,height=640,format=NV12,framerate=30/1' ! \
nvvidconv ! 'video/x-raw,format=I420,width=640,height=640,framerate=30/1' ! \
x264enc bitrate=8000 speed-preset=ultrafast tune=zerolatency key-int-max=30 ! \
h264parse ! qtmux ! \
filesink location=dataset_raw_640.mp4
```

YOLO V11

Run on JetPack 6.1

Installing Ultralytics Package

We will install Ultralytics package on the Jetson with optional dependencies so that we can export the PyTorch models to other different formats.

Update packages list, install pip and upgrade to latest

```
sudo apt update
sudo apt install python3-pip -y
pip install -U pip
```

Install ultralytics pip package with optional dependencies

```
pip install ultralytics[export]
```

Reboot the device

```
sudo reboot
```

Install PyTorch and Torchvision

Install torch 2.5.0 and torchvision 0.20 according to JP6.1

```
pip install
```

```
https://github.com/ultralytics/assets/releases/download/v0.0.0/torch-2.5.0a0+872d972e41.nv24.08-cp310-cp310-linux\_aarch64.whl
```

```
pip install
```

```
https://github.com/ultralytics/assets/releases/download/v0.0.0/torchvision-0.20.0a0+afc54f7-cp310-cp310-linux\_aarch64.whl
```

Install cuSPARSELt to fix a dependency issue with torch 2.5.0

```
wget
```

```
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/arm64/cuda-keyring\_1.1-1\_all.deb
```

```
sudo dpkg -i cuda-keyring_1.1-1_all.deb
```

```
sudo apt-get update
```

```
sudo apt-get -y install libcusparselt0 libcusparselt-dev
```

For JetPack 6 with Python 3.10 support, install onnxruntime-gpu 1.23.0:

```
pip install
```

```
https://github.com/ultralytics/assets/releases/download/v0.0.0/onnxruntime\_gpu-1.23.0-cp310-cp310-linux\_aarch64.whl
```

After ultralytics installation

Install for current user: `pip3 install --upgrade ultralytics`

Verify: `python3 -c "from ultralytics import YOLO; print('OK')"`

```
ls ~/.local/bin | grep yolo (should output yolo)
```

```
echo 'export PATH=$PATH:$HOME/.local/bin' >> ~/.bashrc
```

```
source ~/.bashrc
```

```
yolo help
```

```
pip install numpy==1.23.5 ( numpy should be downgraded to run ultralytics)
```

Convert Model to TensorRT and Run Inference

The YOLO11n model in PyTorch format is converted to TensorRT to run inference with the exported model.

Do-

```
sudo apt install nano -y
```

Create the file: `nano yolov11.py` and paste -

```

from ultralytics import YOLO
import cv2
# 1. Load TensorRT engine
print("[INFO] Loading YOLO11n TensorRT engine...")
model = YOLO("yolo11n.engine", task="detect") # make sure this file is in the same folder
print("[INFO] Engine loaded.")

gst = (
    "nvarguscamerasrc sensor-id=0 ! "
    "video/x-raw(memory:NVMM), width=1280, height=720, format=NV12, framerate=30/1 ! "
    "nvvidconv ! video/x-raw, format=BGRx ! "
    "videoconvert ! video/x-raw, format=BGR ! "
    "appsink drop=1"
)
print("[INFO] Opening camera with GStreamer...")
cap = cv2.VideoCapture(gst, cv2.CAP_GSTREAMER)
if not cap.isOpened():
    print("[ERROR] Failed to open IMX477 via GStreamer. Check pipeline / OpenCV build.")
    raise SystemExit
print("[INFO] Camera opened successfully.")
# 3. Live loop
while True:
    ret, frame = cap.read()
    if not ret:
        print("[ERROR] Failed to read frame from camera.")
        break
    # YOLO11n inference, only 'person' class (0)
    results = model(frame, imgsz=640, device=0, classes=[0])
    annotated = results[0].plot()
    cv2.imshow("YOLO11n TensorRT - IMX477", annotated)
    if cv2.waitKey(1) & 0xFF == 27: # ESC to exit
        break
    cap.release()
    cv2.destroyAllWindows()
print("[INFO] Stopped.")

```

Export YOLO11n → TensorRT engine (FP16)

Run this once:

```
yolo export model=yolo11n.pt format=engine device=0 half=True imgsz=640
```

From ssh:

Fix for Argus / EGL when running from TTY or SSH

```
export DISPLAY=:0
```

```
export XAUTHORITY=/run/user/1000/gdm/Xauthority
```

After building engine:

1) remove pip OpenCV wheels (they override system OpenCV)

```
pip3 uninstall -y opencv-python opencv-contrib-python opencv-python-headless
```

2) install Ubuntu/Jetson OpenCV (built with GStreamer on Jetson images)

```
sudo apt update
```

```
sudo apt install -y python3-opencv
```

3) verify GStreamer is now enabled

```
python3 - << 'PY'
```

```
import cv2
```

```
info = cv2.getBuildInformation()
```

```
print("OpenCV:", cv2.__version__)
```

```
print("cv2 path:", cv2.__file__)
```

```
print("Has GStreamer:", "YES" if "GStreamer:" in info else "NO")
```

```
PY
```

Run script

```
python3 yolo11.py
```

Custom Dataset Training in yolov11 -

<https://youtu.be/r0RspiLG260?si=aeEtRV304jTLxybv>

Pixhawk & Jetson Connection

Hardware Connection (UART)

Pixhawk TELEM2	Jetson Orin Nano
----------------	------------------

TX (Pin 2)	RX (Pin 10)
------------	-------------

RX (Pin 3)	TX (Pin 8)
------------	------------

GND (Pin 6)	GND (Pin 6)
-------------	-------------

5V (Pin 1)	✗ NOT CONNECTED
------------	-----------------

You can to ls on terminal and see

ttyTHS1, which is pixhawk port

And if connected to Serial_port2 that is telem2

Set its protocol to mavlink1 and baud to 115 in mission planner

Hardware Connection (USB)

Directly connect using usb of pixhawk and jetson

You can to ls on terminal and see

ttyACM0 , which is pixhawk port

Jetson Orin System Setup

Enable UART

```
sudo systemctl stop nvgetty
```

```
sudo systemctl disable nvgetty
```

```
sudo usermod -aG dialout $USER
```

```
reboot
```

Install Required Libraries

```
sudo apt-get update && sudo apt-get install -y \  
python3-dev \
```

```
python3-opencv \
```

```
python3-wxgtk4.0 \
```

```
python3-matplotlib \
```

```
python3-lxml \
```

```
libxml2-dev \
```

```
libxslt-dev
```

MAVLink libraries

```
pip3 install pymavlink mavproxy
```

Heartbeat test

```
mavproxy.py --master=/dev/ttyTHS1 --baudrate 115200 (or ttyACM0 if USB connection)
```

MAVLINK - ROUTER :

Using the MAVLink Router on the Jetson allows us to easily operate our Pixhawk using Mission Planner, while ensuring that the Jetson receives all flight and telemetry information in real time.

Instead of connecting telemetry directly to the Pixhawk, the communication is routed through the Jetson. This way, the Jetson can read all MAVLink messages, process them for autonomous decisions, and also send commands when required.

As if we connect the telem to pixhawk, it will block some of the messages to get forwarded to jetson.

MAVLink Router helps the Jetson manage this communication smoothly by forwarding the necessary data between the Pixhawk and Mission Planner without conflicts. As a result, Mission Planner can be used for monitoring and control, and at the same time, the Jetson has full access to all telemetry and flight data for onboard processing.

This setup makes the overall system simpler to operate, more flexible, and better suited for autonomous control.

Install mavlink-router on Jetson

```
sudo apt update  
sudo apt install -y mavlink-router
```

Check it exists:

```
mavlink-routerd -h
```

Create the config file

Create:

```
sudo mkdir -p /etc/mavlink-router  
sudo nano /etc/mavlink-router/main.conf
```

Paste this (edit the serial device + baud to match your setup):

```
[General]
```

```
ReportStats=true
```

```
[UartEndpoint pixhawk]  
Device=/dev/ttyACM0  
Baud=115200
```

```
[UartEndpoint telemetry]  
Device=/dev/ttyTHS1  
Baud=57600
```

```
[UdpEndpoint local]  
Mode=Normal  
Address=0.0.0.0  
Port=14550
```

```
# Script injection endpoint (guarantees forwarding out to telemetry)  
[UdpEndpoint scriptin]  
Mode=Normal  
Address=127.0.0.1  
Port=14650  
AllowBroadcast=true
```

Start & enable the service

```
sudo systemctl enable mavlink-router  
sudo systemctl restart mavlink-router  
sudo systemctl status mavlink-router --no-pager
```

To see logs:

```
journalctl -u mavlink-router -f
```

TMUX :

tmux = a terminal multiplexer: it lets you run multiple shells in one terminal, split panes, and keep programs running even if your SSH disconnects.

When tmux is used

- Keeping a “workspace” open while you disconnect/reconnect with wifi

Install tmux -

```
sudo apt install -y tmux
```

Create a session:

```
tmux new -s drone
```

List sessions:

```
tmux ls
```

Attach to a session:

```
tmux attach -t drone
```

Detach (leave it running):

- **Ctrl+b** then **d**

Kill session:

```
tmux kill-session -t drone
```

Kill all tmux:

```
tmux kill-server
```

But it is good for recording a video or something, but for using mavlink guided commands, you need to set up a system service !

SYSTEM SERVICE :

```
sudo systemctl daemon-reload  
sudo systemctl enable yolov11.service  
sudo systemctl restart yolov11.service  
sudo systemctl stop yolov11.service
```